

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a66c9249d391e22d5.jsonl`  
- SHA-256: `eed95c12e486ee7f707907e52ee2067f3664172e991543cf4720d3f8faa0ac3a`  
- Source modified: `2026-06-10T06:12:18+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:10:10.146Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("reliability") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"Observability report cannot disambiguate WSL-failure vs quota vs empty ? because the segmenters threw away the Failure reason","severity":"high","area":"observability","file":"backend/reporting/harvest.py:137-155 (_ai_stage), backend/reporting/spec.yaml:44-55 (ai_empty_vs_no_rallies)","evidence":"_ai_stage records only `api\_key\_present` (a bool from os.environ) and `segment\_count`; it never receives a Failure with details because process_video returns []. So the spec's only non-key rule, ai_empty_vs_no_rallies, lumps 'provider error, exhausted quota, OR a failed WASB/TrackNet WSL healthcheck' into one generic warn that literally says 'these all return [] silently' (spec.yaml:50-54). silent_provider_noop (spec.yaml:26) only catches the no-key case by re-reading the env var.", "why_it_matters":"The report is the product's promised disambiguation layer, yet for the most common operational failures (dead WSL GPU, blown quota) it can only emit one ambiguous warning and the DB still says status='processed'. The task asks specifically whether the report disambiguates each silent path ? it does not, and the root cause is the contract gap above.", "why":"placeholder"}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:10:18.518Z)

```

1      """Build an observability report for one indexer run.
2
3      Pure-ish harvest: given the validation results/context, the persisted segments,
4      and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5      *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6      so this module only records facts faithfully ? it does not decide warnings.
7
8      Imported only after `backend.reporting` confirms obsreport is installed, so it may
9      import obsreport freely.
10     """
11     from __future__ import annotations
12
13     import os
14     from pathlib import Path
15     from typing import Any, Dict, List, Optional
16
17     try:
18         import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
19     except ImportError:
20         obsreport = None # type: ignore[assignment]
21
22     _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23     _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25
26     def load_spec() -> "obsreport.ReportSpec":
27         return obsreport.load_spec(_SPEC_PATH)
28
29
30     # --- media metadata -----
31     def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32         """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33         distinguishes a real probe from the absence of one (the fallback footgun)."""
34         meta: Dict[str, Any] = {"fps_source": "unknown"}
35         if video_path and os.path.exists(video_path):
36             try:
37                 meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38             except OSError:
39                 pass
40
41         ffmata = (val_context or {}).get("ffprobe_meta") or {}
42         streams = ffmata.get("streams") or []
43         fmt = ffmata.get("format") or {}
44         if streams:
45             s = streams[0]
46             w, h = s.get("width"), s.get("height")
47             meta["width"], meta["height"] = w, h
48             if w and h:
49                 meta["resolution"] = f"{w}x{h}"
50             fps_str = s.get("r_frame_rate", "0/1")
51             try:
52                 num, den = (int(x) for x in fps_str.split("/"))
53                 if den:
54                     meta["fps"] = round(num / den, 2)
55                     meta["fps_source"] = "probed"
56             except (ValueError, ZeroDivisionError):
57                 pass
58             if s.get("codec_name"):
59                 meta["container"] = fmt.get("format_name") or s.get("codec_name")
60         if fmt.get("duration"):
61             try:
62                 meta["duration_s"] = round(float(fmt["duration"]), 1)
63             except (ValueError, TypeError):
64                 pass

```

```

65         if fmt.get("format_name"):
66             meta["container"] = fmt.get("format_name")
67
68         # Audio readiness signal (for Input Quality UX extension per review item 2).
69         # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70         # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71         # We only record presence here (cheap, from fprobe already in context); the actual
72         # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73         # tool for now.
74         audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75         meta["audio_present"] = len(audio_streams) > 0
76         return meta
77
78
79     # --- stage builders -----
80     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81         with rec.stage("validation") as st:
82             total = len(val_results)
83             passed = sum(1 for r in val_results if getattr(r, "passed", False))
84             st.add_metric("checks_passed", passed)
85             st.add_metric("checks_total", total)
86             failures = [r for r in val_results if not getattr(r, "passed", True)]
87             for r in failures:
88                 st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
89             st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92     def _segmentation_stage(
93         rec: "obsreport.RunRecorder",
94         play_segments: List[Dict[str, Any]],
95         candidates: List[Dict[str, Any]],
96         segmenter_requested: Optional[str],
97         segmenter_actual: Optional[str],
98         config_default: Optional[str],
99         was_reingest: bool,
100         ran: bool,
101     ) -> None:
102         with rec.stage("segmentation") as st:
103             if not ran:
104                 st.status = "not_run"
105             seg_count = len(play_segments)
106             total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107             detector_tag = None
108             if play_segments:
109                 detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110             st.add_metric("candidate_windows", len(candidates))
111             st.add_metric("segment_count", seg_count, audience="both")
112             st.add_metric("total_play_s", total_play, unit="s")
113             # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114             # pick it, and it differs from config's default (the genuine surprise case).
115             # An explicit --segmenter override or a halted run is not a divergence.
116             matches_default = True
117             if ran and segmenter_actual and config_default:
118                 matches_default = (segmenter_actual == config_default)
119             st.details.update({
120                 "segmenter_requested": segmenter_requested,
121                 "segmenter_actual": segmenter_actual,
122                 "config_default": config_default,
123                 "segmenter_explicitly_requested": segmenter_requested is not None,
124                 "matches_config_default": matches_default,
125                 "was_reingest": was_reingest,
126                 "detector": detector_tag,
127             })

```

```

128         # Surface metadata trustworthiness at the data level (not just in a warning):
129         # the motion baseline fabricates per-segment metadata; everything else measures
130         it.
131         if ran and segmenter_actual:
132             st.details["metadata_source"] = (
133                 "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134                 else "measured (detector/AI)"
135             )
136
137     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                 play_segments: List[Dict[str, Any]], ran: bool) -> None:
139         """Only meaningful for AI-based segmenters. Records whether the provider key was
140         present ? the signal the silent_provider_noop rule keys on."""
141         ai_based = segmenter_actual in _AI_SEGMENTERS
142         provider = (config.get("ai_provider") or "gemini")
143         key_env = f"{provider.upper()}_API_KEY"
144         with rec.stage("ai_handoff") as st:
145             if not ran or not ai_based:
146                 st.status = "not_run" if not ran else "skipped"
147             st.details.update({
148                 "ai_based": ai_based,
149                 "provider": provider if ai_based else None,
150                 "model": config.get("ai_model") if ai_based else None,
151                 "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
152             })
153             if ai_based:
154                 st.add_metric("confirmed_segments", len(play_segments))
155
156
157     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
158                   video_meta: Dict[str, Any]) -> None:
159         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
160         coverage = None
161         dur = video_meta.get("duration_s")
162         if dur:
163             coverage = round(100.0 * total / dur, 1)
164         notes = []
165         if not play_segments:
166             notes.append("No highlights were produced for this video.")
167         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
168                           coverage_pct=coverage, notes=notes)
169
170
171     # --- top-level -----
172     def record_run(
173         *,
174         video_id: str,
175         video_path: Optional[str],
176         config: Dict[str, Any],
177         val_results: List[Any],
178         val_context: Optional[Dict[str, Any]],
179         play_segments: List[Dict[str, Any]],
180         candidates: List[Dict[str, Any]],
181         segmenter_requested: Optional[str] = None,
182         segmenter_actual: Optional[str] = None,
183         was_reingest: bool = False,
184         segmentation_ran: bool = True,
185         tool_version: Optional[str] = None,
186         spec: Optional["obsreport.ReportSpec"] = None,
187         **recorder_kwargs: Any,
188     ) -> "obsreport.RunRecorder":
189         """Assemble a finalized recorder for an indexer run (does NOT write)."""
190         rec = obsreport.RunRecorder(

```

```

191         "badminton-highlight-indexer",
192         tool_version=tool_version,
193         video_id=video_id,
194         video_id_kind="file_md5",
195         video_path=video_path,
196         spec=spec if spec is not None else load_spec(),
197         **recorder_kwargs,
198     )
199     vm = video_meta_from_context(val_context, video_path)
200     # If we never probed an fps but segmentation still ran, the segmenter must have
201     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
202     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
203     if vm.get("fps_source") == "unknown" and segmentation_ran:
204         vm["fps_source"] = "fallback"
205         if vm.get("fps") is None:
206             vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
207     rec.set_video_meta(**vm)
208
209     _validation_stage(rec, val_results)
210     config_default = (config.get("indexing") or {}).get("default_segmenter")
211     _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
212                         segmenter_actual, config_default, was_reingest,
segmentation_ran)
213     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran)
214     _highlights(rec, play_segments, vm)
215
216     rec.finalize()
217     return rec
218

```

3. user (2026-06-10T06:10:18.818Z)

```

1     # Observability report spec for the badminton-highlight-indexer.
2     # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3     # library; this file is the indexer-specific layout + footgun rules. Every
4     # actionable rule cites a README/FAQ section so a reader can self-debug.
5     name: indexer
6     title: Badminton Highlight Indexer ? Video Report
7
8     # Customer here = whoever receives the highlight reel.
9     customer_verdict:
10         pass: "? Your highlights are ready."
11         pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12         fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14     sections:
15         - { id: run,          title: "Run",          kind: identity,  audience: both }
16         - { id: video,       title: "Video",        kind: video_meta, audience: both }
17         - { id: highlights,  title: "Highlights",    kind: highlights, audience: customer
}
18         - { id: cust_notes, title: "Things to know", kind: flags,      audience: customer
}
19         - { id: stages,     title: "Processing stages", kind: stages,    audience:
technical }
20         - { id: findings,   title: "Findings & warnings", kind: flags,      audience:
technical }
21         - { id: verdict,    title: "Verdict",        kind: verdict,    audience: both }
22
23     rules:
24         # The classic silent failure: 0 segments because the AI provider no-op'd on a
25         # missing API key ? NOT because the video has no rallies.
26         - code: silent_provider_noop

```

```

27     severity: error
28     when:
29       - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
30       - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
31     message: >-
32       0 segments AND no AI provider API key was set. The segmenter (see Processing
33       stages > segmentation > segmenter_actual) hands its candidate windows to the AI
34       provider, which silently did nothing without a key ? so this is a MISSING
35       CREDENTIAL, not 'the video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a
36       .env file in the project root and re-run (see README#Q7).
37     faq_ref: "README#Q7"
38     customer_message: >-
39       We couldn't generate highlights for this video due to a configuration issue
40       on our side. Please contact support.
41
42     # API key WAS present but an AI segmenter still produced nothing ? likely a
43     # provider error / quota / WSL healthcheck issue rather than a truly empty video.
44     - code: ai_empty_vs_no_rallies
45     severity: warn
46     when:
47       - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
48       - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
49       - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
50     message: >-
51       An AI-based segmenter returned 0 segments even though the API key is set.
52       Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
53       failed WSL healthcheck ? these all return [] silently. Re-run with -u for
54       unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
55     faq_ref: "README#Q7"
56
57     # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
58     - code: fps_fallback_used
59     severity: warn
60     when:
61       - { path: "video_meta.fps_source", op: eq, value: fallback }
62     message: >-
63       fps was NOT probed from the file ? a fallback default (shown as the fps value)
64       was used. Because rally timing and motion thresholds scale with fps, boundaries
65       and motion sensitivity may be miscalibrated. This usually means validation didn't
66       run (it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real
67       fps with ffprobe and re-run.
68     faq_ref: "README#Q14"
69
70     # Audio stream missing or disabled. Per VIDEO_RECORDING_GUIDELINES (and ADR-007)
71     # the shuttle "thwack" is a first-class rally cue for recall; recording with audio
72     # on (mic unobstructed, no aggressive noise suppression) is now a hard requirement
73     # for good results with the shuttle-tracking substrates.
74     - code: audio_disabled_or_missing
75     severity: warn
76     when:
77       - { path: "video_meta.audio_present", op: eq, value: false }
78     message: >-
79       No audio stream was detected in the input (or it could not be read). The shuttle
80       "thwack" impact sound is a strong cue that improves rally recall (especially for
81       the WASB/TrackNet detector paths). Record with audio enabled, keep the camera
82       mic unobstructed, and avoid heavy wind/noise-reduction filters that flatten the
83       transient. See VIDEO_RECORDING_GUIDELINES.md (the "Audio (REQUIRED)" row) and
84       AUDIO_RALLY_DETECTION_PLAN.md.
85     faq_ref: "VIDEO_RECORDING_GUIDELINES"
86     customer_message: >-
87       For best highlight results, record with audio on and the mic unobstructed.
88       The shuttle impact sound helps the detector find rallies. Re-record or
89       re-process after enabling audio (see VIDEO_RECORDING_GUIDELINES.md).
90
91     # The motion baseline fabricates its per-segment metadata/events ? they are not

```

```

92     # measured. Warn so nobody trusts them as ground truth.
93     - code: synthetic_motion_metadata
94       severity: warn
95       when:
96         - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
97     message: >-
98       The 'motion' segmenter was used: its per-segment metadata (intensity, speed,
99       events, server/receiver) are PLACEHOLDER values (unmeasured) ? fabricated by the
100       heuristic, NOT derived from the video ? so a PASS with segments here does not mean
101       the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
102     faq_ref: "README#Q10"
103
104     # The segmenter that ran differs from config's default -> you may be looking at a
105     # different detector than expected (config/setup/CLI defaults are known to diverge).
106     - code: segmenter_divergence
107       severity: info
108       when:
109         - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
false }
110         - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
value: false }
111     message: >-
112       The segmenter that ran differs from config.json's default_segmenter, and no
113       explicit --segmenter was given ? so a fallback default kicked in. The indexer's
114       defaults are known to diverge across config.json / setup.py / the CLI; confirm
115       you ran the detector you meant.
116     faq_ref: "README#Q7"
117
118     # Re-processing an already-indexed video replaced its prior segments (INSERT OR
119     # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
120     - code: reingest_replaced_prior
121       severity: info
122       when:
123         - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
124     message: >-
125       This video was already in the database; re-processing REPLACED its previous
126       segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
127       when you intend to re-run; just know the earlier results are gone.
128     faq_ref: "README#Q11"
129
130     # A validation check failed -> segmentation was halted. Point the reader at the
131     # report's Validation section + the per-check FAQ.
132     - code: validation_failed
133       severity: error
134       when:
135         - { path: "stages.validation.status", op: eq, value: error }
136     message: >-
137       One or more validation checks failed, so indexing was halted. See the
138       Processing stages > validation block above: each failed check states the
139       reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
140     faq_ref: "README#Q13"
141     customer_message: >-
142       This video couldn't be processed because it didn't pass our quality checks
143       (for example: too blurry, too low resolution, or not a badminton court).
144

```

4. user (2026-06-10T06:10:32.777Z)

```

backend\pipeline\segmenters\base.py:30:      "no segments found" from "something went wrong"
(missing key, healthcheck
backend\pipeline\audio\hit_detector.py:114:      return []
backend\pipeline\audio\hit_detector.py:120:      return []
backend\pipeline\segmenters\tracknet_hybrid.py:67:      return []
backend\pipeline\segmenters\tracknet_hybrid.py:81:      return []
backend\pipeline\segmenters\tracknet_hybrid.py:86:      return []

```

```

backend\pipeline\segmenters\tracknet_hybrid.py:91:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:116:     # --- Phase 1: env healthcheck (fail
fast with a clear message) ---
backend\pipeline\segmenters\tracknet_hybrid.py:117:     ok, msg = runner.healthcheck()
backend\pipeline\segmenters\tracknet_hybrid.py:120:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:131:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:172:         return []
backend\pipeline\segmenters\tracknet_hybrid.py:186:         return []
backend\pipeline\detectors\base.py:11:- Provide a single place to evolve the healthcheck / error
contract and thread
backend\pipeline\detectors\base.py:21:- healthcheck() -> tuple[bool, str]
backend\pipeline\detectors\base.py:31:2. Implement run_predict and healthcheck (return (bool,
str) tuple).
backend\pipeline\detectors\base.py:41:6. Add unit tests that exercise the contract (isinstance,
healthcheck shape, run
backend\pipeline\detectors\base.py:51:- healthcheck can just check "weights file exists and
torch importable + CUDA?".
backend\pipeline\detectors\base.py:57:The healthcheck message is already logged by the hybrids
on failure. The reporting
backend\pipeline\detectors\base.py:82:     def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\segmenters\wasb_hybrid.py:63:         return []
backend\pipeline\segmenters\wasb_hybrid.py:73:         return []
backend\pipeline\segmenters\wasb_hybrid.py:78:         return []
backend\pipeline\segmenters\wasb_hybrid.py:83:         return []
backend\pipeline\segmenters\wasb_hybrid.py:106:     ok, msg = runner.healthcheck()
backend\pipeline\segmenters\wasb_hybrid.py:109:         return []
backend\pipeline\segmenters\wasb_hybrid.py:118:         return []
backend\pipeline\segmenters\wasb_hybrid.py:152:         return []
backend\pipeline\segmenters\wasb_hybrid.py:166:         return []
backend\pipeline\segmenters\gemini.py:36:     return []
backend\pipeline\segmenters\gemini.py:52:     return []
backend\pipeline\segmenters\gemini.py:58:     return []
backend\pipeline\segmenters\gemini.py:263:     return []
backend\pipeline\segmenters\yolo_hybrid.py:127:         return []
backend\pipeline\segmenters\yolo_hybrid.py:163:         return []
backend\pipeline\segmenters\yolo_hybrid.py:230:         return []
backend\pipeline\segmenters\yolo_hybrid.py:272:         return []
backend\pipeline\segmenters\yolo_hybrid.py:288:         return []
backend\pipeline\segmenters\yolo_hybrid.py:294:         return []
backend\pipeline\segmenters\yolo_hybrid.py:301:         return []
backend\pipeline\segmenters\yolo_hybrid.py:494:         return []
backend\pipeline\detectors\tracknet_runner.py:105:     def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\tracknet_runner.py:120:         return False, "WSL healthcheck
timed out."
backend\pipeline\detectors\tracknet_runner.py:273:         return []
backend\pipeline\detectors\wasb_runner.py:69:     def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\wasb_runner.py:79:         return False, "WSL healthcheck timed
out."

```

5. user (2026-06-10T06:10:34.103Z)

```

docs\EVALUATION.md:38: silently skipped ? so labeling mistakes surface immediately.
docs\GOLDEN_SET_IMPLEMENTATION.md:49:- **Design choice:** `fetch()`/`annotate()` raise loudly on
a missing/incomplete job rather than returning `[]`, so a broken label source can't silently
masquerade as "0 rallies" and skew a regression to green.
docs\BACKLOG.md:80:- ? **Item 3 (Error/Result contract) synergizes with the report.** Today the
report INFERS failures (e.g. `silent_provider_noop` = 0 segments + no API key). A structured
Result/error from providers/segmenters would let the report read the real reason as a first-
class flag. **Revisit:** with item 3.
docs\DEFERRED_HARDENING_PLAN.md:62:eval/regression set. Leakage would silently inflate the very
metrics the product
docs\CODE_MAP.md:51: -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
docs\CODE_MAP.md:94: - `::AppConfig` ? root. Scalars `sport='badminton'`,
`ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections

```

```

`validation`,`indexing`,`output`,`stitching`(`default_factory`). `model_config` =
{extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys
silently kept (legacy tolerance).
docs\CODE_MAP.md:108:[Omitted long matching line]
docs\CODE_MAP.md:113:- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name
`'gemini'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5`
hardcoded; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates;
`parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids
feed the fine-tuning table).
docs\CODE_MAP.md:121:[Omitted long matching line]
docs\CODE_MAP.md:130:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI.
`::BASELINE=(0.02,2.0,2.0)`, `::default_grid` (45 cfigs), `::make_predict_fn` (parses trajectory
once, closes over `trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time`
cols; ? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ?
`{load_golden_gts, make_predict_fn, BASELINE}` imported by 4 downstream drivers
(calibrate_local, export_wasb_segments, gemini_refine, audio/e1_probe).
docs\CODE_MAP.md:142:[Omitted long matching line]
docs\CODE_MAP.md:150:- `@backend/tools/rally_slicer.py` ? standalone CLI.
`::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently
skips degenerate rows), `::slice_rallies`, `::ClipWindow` (rally:str),
`::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy). `_read_time` ->
backend/eval/annotations.parse_timestamp`.
docs\CODE_MAP.md:166:[Omitted long matching line]
docs\CODE_MAP.md:173:[Omitted long matching line]
docs\CODE_MAP.md:178:[Omitted long matching line]
docs\CODE_MAP.md:202:- golden/slicer (`calibrate_wasb.load_golden_gts`,
`rally_slicer.load_rally_labels`, `export.SEG_FIELDS`):
`rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`; SILENTLY skips bad rows.
docs\CODE_MAP.md:239:- `**AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{ }).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
docs\CODE_MAP.md:241:- `**Gemini free-tier / API-key dead path:** hybrids + gemini return ``[]`
(not error) if `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op.
`analyze_video` also returns ``[]` on real failure -> empty != "no rallies". The reporting
`silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
the report.
docs\CODE_MAP.md:242:- `**Reporting never breaks a run:** `emit_indexer_report` swallows all
exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure.
`obsreport` is a SOFT dep ? absent -> no-op.
docs\CODE_MAP.md:243:- `**spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-
navigate harvest's stage/metric/detail names; rename a detail in harvest.py and the matching
rule silently stops firing. Add rules in spec.yaml, not code.
docs\CODE_MAP.md:244:- `**fps / frame_width must be PROBED not assumed:** `_probe_fps_width` /
runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes
`59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
docs\GOLDEN_SET_PHASE1_VIDEOS.md:78:> `**Guardrails ? TODO (design with the user):** train and
eval live in `**separate catalogs**` today, kept apart by convention. We want hard guardrails ?
permissions / explicit approval / re-confirmation at ingest & export ? so a video can `**never**`
silently land in both. Tracked in BACKLOG.md; to be designed in a later session.
docs\VIDEO_RECORDING_GUIDELINES.md:58:2. `**Programmatic pre-flight check**` ? extend
`backend/validators/` to `*reject or warn*` on out-of-spec uploads at ingest
(resolution/fps/blur/framing), so the product fails fast with actionable feedback instead of
producing poor highlights silently.
docs\archives\decisions\DECISIONS.md:123: download URLs and silently drops to 144p (yt-dlp
#12482). yt-dlp's `default` client
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:197:- `**Item 3
(Error/Result contract) ? synergy.** A structured Result from providers/segmenters lets the
report read failure reasons directly instead of inferring them (e.g. `silent_provider_noop`).
```

6. user (2026-06-10T06:10:48.876Z)

- 1 """"Abstract base for detector runners (TrackNet, WASB, future native, etc.).
- 2
- 3 This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
- 4 of the post-observability re-prioritization) to de-risk the WSL/conda dependency for

```

5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7     The ABC lets us:
8     - Keep the current WSL adapters working unchanged for Windows users.
9     - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10    the hybrid segmenters or the model-agnostic windowing brain.
11    - Provide a single place to evolve the healthcheck / error contract and thread
12    detector readiness into per-video reports.
13
14    Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15    - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16      Run detector, return the Windows-side path to the produced trajectory CSV
17      (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18      The caller (hybrid) is responsible for output dir; impls may stage the video
19      into WSL fs for perf.
20
21    - healthcheck() -> tuple[bool, str]
22      Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23      Message is logged (and can be surfaced to reports or UX in future).
24      Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26    Existing behavior is 100% preserved. The only additions are the inheritance and
27    the explicit common type for future implementers.
28
29    How to add a new runner (for docs / future contributors):
30    1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31    2. Implement run_predict and healthcheck (return (bool, str) tuple).
32    3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33       TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34       deliberately static and model-agnostic). If the new detector needs different
35       windowing, it can implement its own or we generalize later.
36    4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37       registry never pulls torch/tf on a machine that doesn't have them).
38    5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39       DetectorRunner instances (they already do the right thing via duck-typing + the
40       type hints we add here).
41    6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43    7. No change to eval/calibration code paths (they consume the CSV + the static
44       windowing helpers directly).
45
46    Optional Linux-native path (the main de-risk goal):
47    A LinuxNativeRunner (or ONNXRunner) would:
48    - Skip all wsl / _stage / conda activate.
49    - Load weights locally (torch or onnx) or call a native binary.
50    - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51    - healthcheck can just check "weights file exists and torch importable + CUDA?".
52    This removes the WSL requirement for Linux users / future SaaS workers while
53    keeping the exact same candidate quality characteristics (or better, once native
54    weights are available).
55
56    Error surfacing into reports:
57    The healthcheck message is already logged by the hybrids on failure. The reporting
58    harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59    segmenter Results (or pass detector_health into emit_indexer_report context), the
60    (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61    stage. The ABC makes that future change local to one place.
62
63    See also:
64    - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65    - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66    - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67    """
68
69    from abc import ABC, abstractmethod

```

70 from typing import Optional, Tuple

7. user (2026-06-10T06:10:50.249Z)

9:remaining high-value hygiene items not already done (config/structure/Result/logging/

8. user (2026-06-10T06:11:02.168Z)

1:# Deferred Hardening Plan ? Train/Eval Guardrails, Async Jobs, Packaging
13:#13 Train/eval guardrails (correctness of the data flywheel) ? #16 Async jobs
18:## 1. Packaging modernization (debt #7)
20:### Problem
28:### Approach
43:### Risks
49:### Verification
55:## 2. Train/eval separation guardrails (debt #13)
57:### Problem
65:### Approach
78:### Risks
84:### Verification
90:## 3. Async job model (debt #16)
92:### Problem
96:### Approach (minimal first slice ? single self-hosted box)
107:### Risks
115:### Verification
121:## Out of this doc

9. user (2026-06-10T06:11:08.539Z)

60 - ? **Approved directory structure** ? Adopt a layered approach under `backend/` for professionalism and future growth (no flat dumping of new modules, no explosion of 15+ single-file dirs):
61 - `backend/config/` (package ? start with this)
62 - `backend/api/` (FastAPI app, routes, static UI)
63 - `backend/pipeline/` (validators + segmenters + stitcher + audio + detectors)
64 - `backend/infrastructure/` (database + external adapters)
65 - Keep `backend/eval/` prominent as a first-class concern.
66 - Logical grouping for providers, sports, annotations, utils, and a small `core/` only when needed.
67 - ? **Sequencing (Config first, then atomic refactor)** ? Implement Deliverable 1 (Config) **first**, introducing the `backend/config/` package immediately. Perform the full directory structure refactor as a **separate high-pri atomic PR** afterwards. This minimizes conflicts, keeps PRs focused, and gives Claude a clean window with no concurrent moves.
68 - ? **Deliverable 1 (Config)** ? `backend/config/` package (`models.py` + `loader.py`) instead of a single `config.py`. Typed `AppConfig`, central loader with built-in defaults + file override, `extra="allow"` for migration. Update `setup.py` + `main.py`. Add tests. Land this before the layout move.
69 - ? **Subsequent items** ? See the refreshed `LOW\_REGRET\_MOVES\_PLAN.md` for the full ordered list (Error contracts, logging, packaging, detector abstraction, etc.). The structure refactor is now explicitly tracked as its own high-pri step right after Config.
70
71 **Status legend reminder:** ? = approved / in progress tracking, ? = do when triggered.
72
73 ## Observability reports ? shipped; intersections with the low-regret items (2026-06-07)
74 *(source: per-video observability feature ? indexer PR #41/#42/#43, collector PR #11/#12; shared lib `obsreport`)*
75
76 - ? **Low-regret items 1 (Config pkg, PR #39) + 2 (directory restructure, PR #40) are DONE.** Remaining = items 3?9 in `LOW\_REGRET\_MOVES\_PLAN.md`.
77 - ? **Per-video observability reports shipped in BOTH tools** (a TECHNICAL report + a CUSTOMER-friendly summary), via the **soft-dependency** lib `obsreport` (github.com/avidullu/sports-obsreport, MIT, v0.1.3). Each run writes `78 - ? **Item 8 (Input-Quality UX) is LARGELY SUBSUMED** by these reports (validation stage

```

+ footgun flags + verdict already surface quality; a clean customer summary already exists).
**Do NOT build a parallel mechanism** ? instead EXTEND the report: add an audio-readiness
stage/flag to `backend/reporting/` (harvest) + the obsreport spec. **Revisit:** when item 8 is
picked up.
79     - ? **Item 7 (unify hybrid segmenters) should land before the report's real AI-provider
timing.** The report records AI timing as `not_captured`; once `wasb_hybrid`/`tracknet_hybrid`
are unified there is ONE place to thread a recorder callback through `provider.analyze_video` ?
the report. **Revisit:** with item 7 (Claude to add the timing hook there).
80     - ? **Item 3 (Error/Result contract) synergizes with the report.** Today the report
INFERS failures (e.g. `silent_provider_noop` = 0 segments + no API key). A structured
Result/error from providers/segmenters would let the report read the real reason as a first-
class flag. **Revisit:** with item 3.
81     - ? **`obsreport` is a SOFT/commented dep** in `requirements.txt` (works locally via
editable install; pipeline no-ops if absent). **Pin the `git+ssh@v0.1.3` tag** in both repos
once SSH/CI access to the private lib repo exists.
82     - ? **`run_all_tests.py`** added at repo root ? run the full suite (clear PASS/FAIL,
pytest exit code) before sending PRs.
83     - ? **Config-migration follow-ups (DONE ? config & identity consistency pass)** ? the
two typed-config bugs from
84     #39/#40 are fixed: (1) **`/api/compile` now honors `--output-dir`** and (2)
**`output_dir` is a real field on
85     `OutputConfig`** (default `"output"`); the CLI overrides it on the typed model in
`main()`, retiring the
86     write-only `_runtime_overrides` hack. Same pass also unified `video_id` MD5 derivation
into
87     `backend/utils/hashing.py::compute_video_id` and migrated all four `run*.py` scripts
off raw `json.load` onto
88     `backend.config.load_config`. See `CODE_MAP.md` §2.0/§6 (footguns retired).
89

```

10. assistant (2026-06-10T06:11:30.509Z)

The finding is factually accurate but appears to be a documented deferral. Let me quickly verify the one possibly-new consequence claimed ? DB `status='processed'` on empty results.

11. user (2026-06-10T06:11:31.953Z)

```

<tool_use_error>Path does not exist: C:\Users\avidu\Projects\badminton-highlight-
indexer\main.py. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer.</tool_use_error>

```

12. user (2026-06-10T06:11:37.358Z)

```

backend\main.py:123:         db_instance.add_video(video_id, req.video_path, "processed",
[r.dict() for r in val_results])
backend\main.py:368:         status = "failed" if failures else "processed"

```

13. user (2026-06-10T06:11:44.040Z)

```

350         db_instance = Database(db_path)
351
352         # CLI processing mode (no web UI needed)
353         if args.video_path and not args.server:
354             print(f"[CLI] Processing video file: {args.video_path}")
355             if not os.path.exists(args.video_path):
356                 print(f"[FAIL] Video file not found: {args.video_path}")
357             return
358
359             video_id = compute_video_id(args.video_path)
360             was_reingest = db_instance.get_video(video_id) is not None
361
362             # Validate (with-context variant surfaces ffprobe_meta for the report)
363             print("[CLI] Validating...")
364             val_results, val_context = registry.run_all_with_context(args.video_path,

```

```

global_config)
365         failures = [r for r in val_results if not r.passed]
366
367         # Save video status
368         status = "failed" if failures else "processed"
369         db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])
370
371         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
372         segmenter_actual = None
373         segmentation_ran = False
374         try:
375             print("\n" + "="*40)
376             print("VALIDATION SUMMARY")
377             print("="*40)
378             for r in val_results:
379                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
380                 print(f"{status_symbol} {r.validator_name}: {r.message}")
381             print("="*40 + "\n")
382
383             if failures:
384                 print("[FAIL] Video failed checks. Indexing halted.")
385                 return
386
387             # Index
388             segmenter_cls = segmenter_registry.get(seg_name)
389             if not segmenter_cls:

```

14. user (2026-06-10T06:12:14.281Z)

Structured output provided successfully

15. assistant (2026-06-10T06:12:18.497Z)

Verification complete.